Section 4.5: Beyond Classification

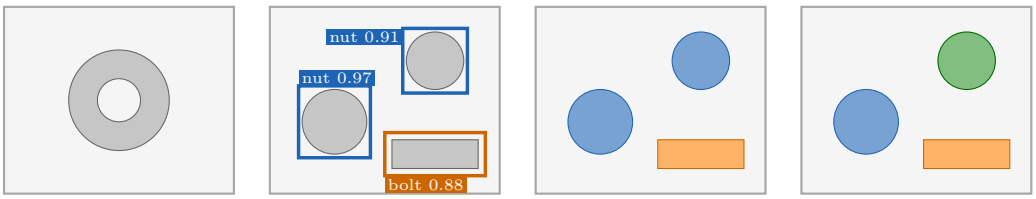
Object Detection, Segmentation and OCR — an Overview

AI-IM Class — Lecture Notes

So far a CNN has answered one question per image: *what is in it?* Many real problems need more. A robot must know *where* each part lies; an inspection system must outline the scratch, not only report that there is one; a traceability system must *read* the serial number. The same convolutional feature extractor (Sections 4.2–4.4) handles all of these. Only the head and the loss change. This section is an overview; the details of modern detectors and text recognisers are left to an advanced topic. We

- compare the main computer-vision tasks,
- learn the basic tools of object detection (boxes, IoU, non-maximum suppression) and the idea behind YOLO,
- see how segmentation predicts a class for every pixel,
- outline how OCR reads text, and
- map these tasks to manufacturing applications.

1 A Family of Vision Tasks



label: `nut` | boxes + labels | class of every pixel | pixels of every object

(a) classification **(b) object detection** **(c) semantic segm.** **(d) instance segm.**

Figure 1: Four vision tasks on images of parts. (a) One label for the whole image. (b) A box, class and confidence score for every object. (c) Each pixel is coloured by class; the two nuts get the same colour. (d) Each object gets its own mask, so the two nuts are distinguished.

Task	Output per image	Loss (main parts)
Classification	C class scores	cross-entropy
Object detection	a list of (box, class, score)	box regression + cross-entropy
Semantic segmentation	a class for every pixel: (C, H, W) scores	cross-entropy per pixel
Instance segmentation	detection + a pixel mask per object	detection loss + mask loss

Notice that every loss is built from the two pieces of Section 3.4: a *regression* loss (MSE or similar) for continuous outputs such as box coordinates, and *cross-entropy* for class labels.

2 Object Detection

2.1 Bounding Boxes

An object's location is a **bounding box**, the smallest axis-aligned rectangle around it. Two formats are common (pixel coordinates are measured from the top-left corner, y pointing down):

- **corners** $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$;
- **centre and size** (x_c, y_c, w, h), often divided by the image width and height so that all four numbers lie in $[0, 1]$. YOLO uses this form.

Exercise 1 Convert a box

An image is 200 pixels wide and 120 pixels tall. A box has corners $(20, 30, 60, 90)$.

(a) In pixels: $x_c = 40$, $y_c = 60$, $w = 40$, $h = 60$.

(b) Normalised: $\big(0.2,\ 0.5,\ 0.2,\ 0.5\big)$.

2.2 Intersection over Union

How do we decide whether a predicted box is "right"? It will never match the true box exactly. The standard measure of overlap is the **intersection over union**:

$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)} = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A) + \mathrm{area}(B) - \mathrm{area}(A \cap B)}.$$

IoU $= 1$ for identical boxes and 0 for boxes that do not touch. A detection usually counts as correct if IoU ≥ 0.5 with a true box of the same class.

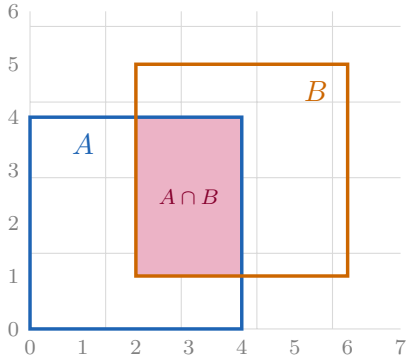


Figure 2: Boxes $A = (0, 0, 4, 4)$ and $B = (2, 1, 6, 5)$ in corner format. Their intersection is shaded.

Exercise 2 Compute IoU

(a) For the boxes in Figure 2: the intersection is 2 wide and 3 tall, so its area is 6. The union has area $16 + 16 - 6 = 26$, and IoU $= 6/26 \approx 0.23$.
(b) Move B to $(1, 1, 5, 5)$. Now the intersection is 3×3, and IoU $= 9/23 \approx 0.39$.
(c) In general, the intersection of boxes A and B in corner format has width $\max\big(0,\ \min(x^A_{\max}, x^B_{\max}) - \max(x^A_{\min}, x^B_{\min})\big)$, and similarly for its height.

2.3 Non-Maximum Suppression

A detector looks at many positions, so it usually reports the same object several times with slightly different boxes. **Non-maximum suppression (NMS)** keeps the best box for each object:

1. Sort all boxes of one class by confidence score.
2. Keep the box with the highest score.
3. Delete every remaining box whose IoU with the kept box is above a threshold (e.g. 0.5): they are duplicates of it.
4. Repeat steps 2–3 with the boxes that are left.

Exercise 3 Run NMS by hand

A detector reports four "nut" boxes with scores A: 0.90, B: 0.80, C: 0.75, D: 0.60. Their pairwise IoU is

	A	B	C	D
A	—	0.70	0.10	0.05
B	0.70	—	0.20	0.10
C	0.10	0.20	—	0.60
D	0.05	0.10	0.60	—

Use threshold 0.5.
(a) Round 1: keep A; delete B (IoU 0.70).
(b) Round 2: keep C; delete D (IoU 0.60).
(c) The final detections are A, C: the image contains 2 nuts.

2.4 Two Families of Detectors

- **Two-stage detectors** (R-CNN 2014, Fast R-CNN, Faster R-CNN 2015). Stage 1 proposes a few thousand regions that may contain an object; stage 2 runs a CNN classifier and box regressor on each one. They are accurate but slower.
- **One-stage detectors** (YOLO 2016, SSD). A single forward pass of one network predicts all boxes and classes at once, fast enough for live video. Today's YOLO versions are both fast and accurate, which is why they are the usual first choice in industry.

2.5 The Idea Behind YOLO

YOLO ("You Only Look Once", Redmon et al.) turns detection into a single regression problem. The image is divided into an $S \times S$ grid. The grid cell that contains the *centre* of an object is responsible for detecting it. Every cell predicts:

- B boxes, each with 5 numbers: (x_c, y_c, w, h) and a *confidence* that the box contains an object;
- C class probabilities (which class the object is, if there is one).

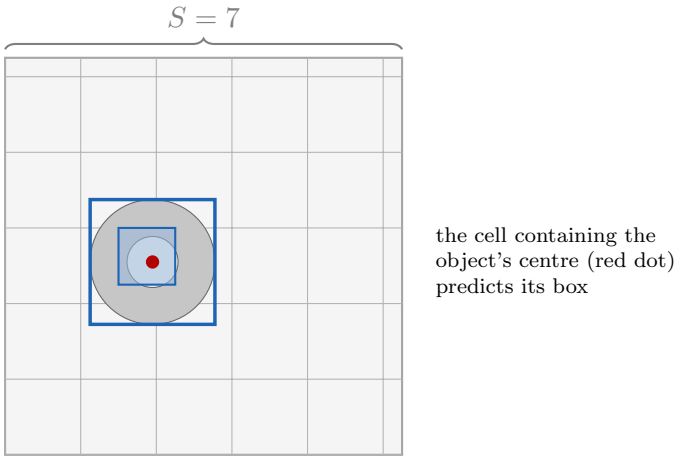


Figure 3: YOLO divides the image into an $S \times S$ grid. The shaded cell contains the centre of the nut and is responsible for its box.

The whole output is therefore one tensor of shape

$$S \times S \times (5B + C).$$

Afterwards, boxes with low confidence are discarded and NMS removes the duplicates.

Exercise 4 Size of the YOLO output

(a) The original YOLO used $S = 7$, $B = 2$ and the $C = 20$ classes of the PASCAL VOC dataset. Each cell predicts $5 \cdot 2 + 20 = 30$ numbers, and the output has $7 \cdot 7 \cdot 30 = 1\,470$ numbers.

(b) A defect detector uses $S = 13$, $B = 5$ and $C = 3$ defect types. The output shape is $13 \times 13 \times 28$.

Modern versions refine this idea (several grid sizes for large and small objects, better box parametrisations, better losses), but the principle is unchanged: one CNN, one pass, a fixed-size output tensor. In practice students and engineers use a pretrained model from a library:

```python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")              # small model pretrained on COCO
results = model("conveyor.jpg")         # boxes, classes, scores (after NMS)
model.train(data="defects.yaml", epochs=50)   # fine-tune on our own data
```

2.6 Measuring a Detector

A predicted box is a **true positive** if it has IoU ≥ 0.5 with a true box of the same class (each true box can be matched only once); otherwise it is a **false positive**. True boxes that no prediction matches are **missed**.

$$\text{precision} = \frac{\text{true positives}}{\text{predicted boxes}}, \qquad \text{recall} = \frac{\text{true positives}}{\text{true boxes}}.$$

Precision asks "when the detector says defect, is it right?"; recall asks "how many of the defects did it find?".

Exercise 5 Precision and recall

An image set contains 10 scratches. The detector reports 8 boxes, of which 6 match a scratch with IoU ≥ 0.5.

$$\text{precision} = 6/8 = 0.75, \qquad \text{recall} = 6/10 = 0.60.$$

Raising the confidence threshold usually makes precision higher and recall lower.

Sweeping the confidence threshold traces a precision–recall curve. The area under it is the **average precision** (AP) of a class, and its mean over all classes is the **mAP**, the standard single-number score of a detector.

3 Segmentation

Semantic segmentation classifies every pixel. For an $H \times W$ image and C classes the network outputs a (C, H, W) tensor of logits: a softmax over the C classes *at each pixel*, trained with the cross-entropy averaged over all pixels.

A classification CNN shrinks the feature maps to gain a large receptive field, but segmentation needs the answer back at full resolution. The solution is an **encoder–decoder**: the encoder is an ordinary CNN that downsamples; the decoder upsamples again (for example with *transposed convolutions*, which enlarge a feature map). The popular **U-Net** (2015) adds skip connections that copy fine detail from each encoder level to the decoder level of the same size.

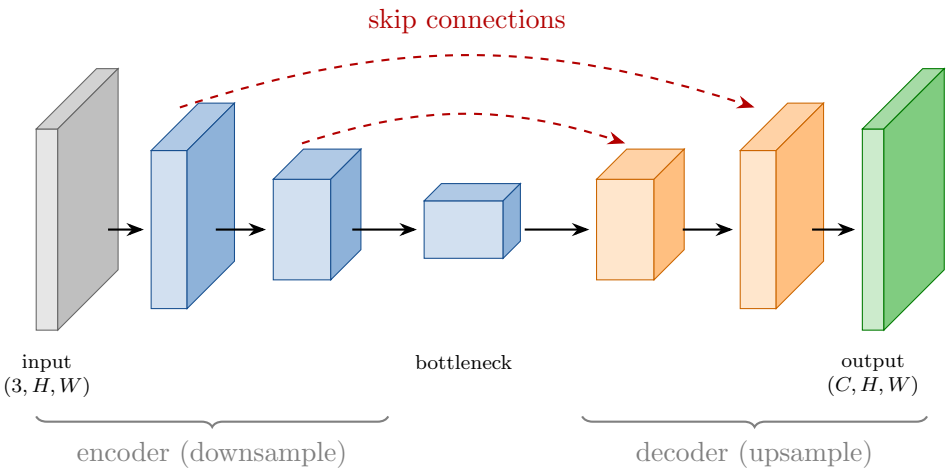


Figure 4: An encoder–decoder for segmentation in the style of U-Net. The output has one score map per class, at the input resolution.

Exercise 6 Segmentation output

A crack-inspection network segments 256×256 images into $C = 3$ classes (background, crack, stain).

(a) The output tensor for one image has shape $(3, 256, 256)$.

(b) The loss averages the cross-entropy over $256^2 = 65\,536$ pixel predictions.

Instance segmentation (e.g. Mask R-CNN, 2017) combines both ideas: a detector finds each object, and a small segmentation head predicts a mask inside each box.

4 Optical Character Recognition (OCR)

OCR turns text in an image into a string: a serial number on a part, a lot code and expiry date on a package, the reading of a meter. Modern OCR systems are a pipeline of the tasks above.

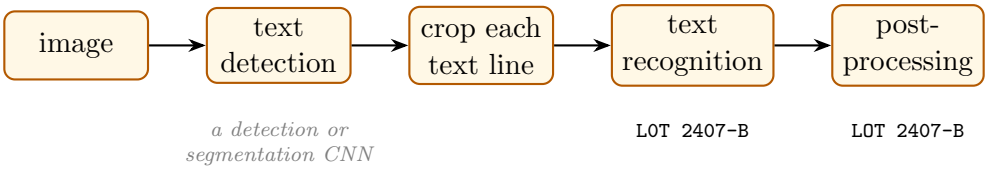


Figure 5: An OCR pipeline. Post-processing applies known rules (here: the code starts with the word `LOT`, so the digit `0` must be the letter `O`).

1. **Text detection** finds the regions that contain text. This is object detection or segmentation, as described above, often with rotated boxes.
2. **Text recognition** reads one cropped line. The classic approach cuts the line into single characters and classifies each with a CNN, exactly like MNIST digits. This fails when characters touch or are unevenly spaced. Modern recognisers such as **CRNN** instead run a CNN over the whole line, read the resulting feature map column by column from left to right as a sequence, and predict one character (or

"blank") per column. They are trained with the **CTC** (connectionist temporal classification) loss, which does not need to know where each character is.

3. **Post-processing** corrects the string with known formats, dictionaries or checksums.

Exercise 7 Decoding a CTC output

A CTC recogniser predicts one symbol per column; "–" is the blank. To get the text, first *merge repeated symbols*, then *remove blanks*.

(a) `- 2 2 - 4 4 4 - 0 7 7` → 2407

(b) `- 1 1 - 1 - 5 5` → 115

(c) Why is the blank needed? Without it, "11" would merge into a single "1".

Ready-made OCR libraries include **Tesseract**, **EasyOCR** and **PaddleOCR**. For new projects, transformer-based recognisers and large vision–language models are increasingly used as well.

5 Vision in Manufacturing

Application	Task	Example
Accept / reject a part	classification	surface defect type on steel sheet
Locate defects or count parts	object detection	missing components on a PCB; parts on a conveyor
Measure defect shape and size	segmentation	crack or weld-seam outline
Robot picking	detection / instance segm.	grasp points in a bin of parts
Traceability	OCR	serial numbers, lot codes, expiry dates
Rare or unknown defects	anomaly detection	train only on good parts, flag anything unusual

Practical advice. Start from a pretrained model and fine-tune it (transfer learning, Section 4.4); a few hundred well-labelled images per class are often enough. Invest in consistent lighting and camera position: a controlled image makes every task easier. Choose the evaluation measure with the cost of errors in mind: a missed defect (low recall) is usually far more expensive than a false alarm.

6 Summary

- Classification, detection, semantic and instance segmentation share a CNN feature extractor; they differ in the head and the loss.

- A detection is a box, a class and a score. IoU measures box overlap; NMS removes duplicate boxes.
- YOLO predicts all boxes in one pass as a tensor of shape $S \times S \times (5B+C)$. Two-stage detectors propose regions first, then classify them.
- Detectors are evaluated by precision, recall and mAP, with matches defined by IoU ≥ 0.5.
- Segmentation predicts a class per pixel with an encoder–decoder such as U-Net.
- OCR is a pipeline: detect text, recognise each line (CNN features, read as a sequence, CTC), then post-process.

Advanced topics. The YOLO loss function and anchor boxes, the details of CTC, transposed convolutions and modern segmentation models, and anomaly detection will be covered in a later topic.